COMP 1020 Fall 2026 - Assignment 3

Material covered: Recursion, sorting, multidimensional arrays

Echoes and the Signal Wall consists of two recursive problems and one multiplayer game. Implement the required methods exactly as specified and follow the COMP 1020 Programming Standards.

Problem 1: Recursive Echoes (10 marks)

A communication system generates nested messages according to:

```
echo(1) = "(1)"
echo(n) = "(" + echo(n-1) + n + echo(n-1) + ")"
    echo(2) -> ((1)2(1))
    echo(3) -> (((1)2(1))3((1)2(1)))
```

Implement in A3Recursion.java:

```
String echo(int n)
```

Return an empty string unless 1 <= n <= 9. Use recursion, with no loops.

Problem 2: Travel Itineraries (15 marks)

Generate all ordered pairs of different destinations, each combined with every transportation method. For the arrays below, two of the 12 required output lines are:

```
places = {"Paris", "Rome", "Tokyo"}
transport = {"Train", "Plane"}
Paris -> Rome by Train
Rome -> Paris by Train
```

Implement in A3Recursion.java:

```
void printItineraries(String[] places, String[] transport)
```

Print one itinerary per line and no other output. For p places and t transport methods, produce p*(p-1)*t lines in any order. CodeGrade will compare output lines without regard to order. Array entries are assumed unique and non-null.

If either array is null, places has fewer than 2 entries, transport is empty, places has more than 10 entries, or transport has more than 5 entries, print nothing. Recursion only, no loops; private recursive helpers are permitted. No-loops requirements in Problems 1 and 2 will be checked manually.

Problem 3: The Signal Wall (75 marks)

Players compete to connect N of their own signals horizontally, vertically, or diagonally on a rectangular wall stored as int[][] (rows first, columns second).

Game setup

Read four comma-separated integers in order: players (2-9), rows (3-12), columns (3-25), N (3 to max(rows, columns)). Example: 3,4,6,3 creates 3 players and a 4-row, 6-column wall with N = 3.

Gameplay
Player 1 moves first, followed by Players 2 through the final player, repeating in order. On each turn, select a row numbered from 1. Place the current player number in its leftmost empty cell (0 denotes empty).
If that row is full, discard its leftmost signal, shift the other signals one cell left, and insert the new signal at the right:

```
Before:  1  2  1  3  2
After:   2  1  3  2  3   (Player 3 played)
```

After each valid move, check every player. A win is N or more matching signals in a horizontal, vertical, or either diagonal direction; multiple players may win at once. There is no draw because full rows can always shift. Stop on a win, or on end of input (EOF) as specified below.

Required static methods

Implement all methods below as public static methods in A3Main.java. Invalid walls are null, have zero rows, have a null row, have zero columns, or have rows of unequal lengths. Such walls cause printWall to print nothing, and other wall methods to return false or an empty array. Any nonzero cell is occupied.

```
String parseSetup(String input, int[] result)
```

Accept exactly four comma-separated integers; trim whitespace around each value. A leading + is allowed; missing/empty fields (including a trailing comma), nonintegers, out-of-range values, null input, and a result array that is null or not of length 4 are invalid. Return null and fill result only when all four values are valid; otherwise return a descriptive error and leave result unchanged.

```
void printWall(int[][] wall)
```

Print a column header and one line per row. Use a width of 3 characters, right-aligned, for each label/cell; begin the header with three spaces. Print 0 as a period and nonzero integers as numbers. No blank lines, extra text, or trailing spaces. All cell values in printWall tests will be between -9 and 99 (at most two characters). Example for 12 columns (first row shown):

```
          1  2  3  4  5  6  7  8  9 10 11 12
       1  .  .  .  .  .  .  .  .  .  .  .  .
```

```
boolean placeSignal(int[][] wall, int player, int row)
```

Place a signal, including full-row shifting. Return false without changing the wall if its shape is invalid, player is outside 1-9, or the 1-based row is outside the wall.

```
boolean checkWin(int[][] wall, int player, int n)
```

Return true if the player has at least n consecutive signals in any of the four directions; otherwise false. Return false for an invalid wall, a player outside 1-9, or n outside 1-max(rows, columns).

```
int[] rankPlayers(int[][] wall, int numPlayers)
```

Return player IDs 1 through numPlayers sorted by decreasing count of their signals on the wall. Ties are sorted by increasing player ID, including players with zero signals. Ignore cell values outside 1-numPlayers when counting. Return an empty array for an invalid wall or numPlayers outside 2-9. Use insertion sort or selection sort (no built-in sorting).

Main program

Implement public static void main(String[] args) in A3Main. Print
"Enter players,rows,columns,N: "
for setup and
"Player X row: "
for moves (X is the player number). On invalid setup, print "Invalid setup: " followed by the explanation and prompt for setup again; on invalid row input, print "Invalid row." and retry the same player. Input is read from System.in.

Display the wall initially and once after every successful move, including the final move (do not display the winning wall twice). After a winning move, print "Player X wins!" for each winner in player-number order; then print "Leaderboard:", followed by "Player X: Y" for each ranked player (Y is their remaining signal count), and finally "Game over."

If input ends at any prompt, print "Input ended." and exit normally. Do not print "Game over.", a winner announcement, or a leaderboard at EOF. Do not loop indefinitely or throw an exception at EOF. Game I/O will be tested manually or with lenient matching; automated tests will check the specified public methods. Automated game tests will supply moves that reach a win.

Sample run (3 x 3 wall)

```
Enter players,rows,columns,N: 2,3,3,3
     1  2  3
  1  .  .  .
  2  .  .  .
  3  .  .  .
Player 1 row: 1
     1  2  3
  1  1  .  .
  2  .  .  .
  3  .  .  .
Player 2 row: 2
     1  2  3
  1  1  .  .
  2  2  .  .
  3  .  .  .
Player 1 row: 1
     1  2  3
  1  1  1  .
  2  2  .  .
  3  .  .  .
Player 2 row: 2
     1  2  3
  1  1  1  .
  2  2  2  .
  3  .  .  .
Player 1 row: 1
     1  2  3
  1  1  1  1
  2  2  2  .
  3  .  .  .
Player 1 wins!
Leaderboard:
Player 1: 3
Player 2: 2
Game over.
```

Submission

Submission: Submit only A3Recursion.java and A3Main.java to CodeGrade. You may add private helpers. No external input files or file I/O are required. Follow the assignment guidelines and academic-integrity rules posted on UMLearn; submission deadlines and late policies are also posted there. Use a single input reader on System.in.